

Intermediate Operations

Operation	Beschreibung	Eingabe	Ausgabe	Beispiel
<code>.filter()</code>	Filtert Elemente nach einer Bedingung	<code>Stream<Integer></code>	<code>Stream<Integer></code> mit gefilterten Werten	<code>Stream.of(1,2,3,4).filter(n -> n > 2) → [3,4]</code>
<code>.map()</code>	Wandelt jedes Element in ein anderes Objekt um	<code>Stream<Person></code>	<code>Stream<String></code> (z. B. Namen)	<code>persons.stream().map(p -> p.name()) → ["Bob", "Alice", "Jane"]</code>
<code>.flatMap()</code>	Vereinfacht verschachtelte Strukturen („flachklopfen“)	<code>Stream<List<Person>></code>	<code>Stream<Person></code>	<code>teams.stream().flatMap(List::stream)</code>
<code>.sorted()</code>	Sortiert Elemente nach Comparator	<code>Stream<Integer></code>	<code>Stream<Integer></code> sortiert	<code>Stream.of(3,1,2).sorted() → [1,2,3]</code>
<code>.distinct()</code>	Entfernt doppelte Elemente	<code>Stream<Integer></code>	<code>Stream<Integer></code> ohne Duplikate	<code>Stream.of(1,1,2).distinct() → [1,2]</code>
<code>.skip(n)</code>	Überspringt die ersten n Elemente	<code>Stream<String></code>	<code>Stream<String></code> mit Rest	<code>Stream.of("A", "B", "C").skip(1) → ["B", "C"]</code>
<code>.limit(n)</code>	Begrenzt auf n Elemente	<code>Stream<String></code>	<code>Stream<String></code> (erste n)	<code>Stream.of("A", "B", "C").limit(2) → ["A", "B"]</code>
<code>.peek()</code>	Führt Aktion mit jedem Element aus (z. B. Logging), verändert nichts	<code>Stream<T></code>	<code>Stream<T></code> (unverändert)	<code>stream.peek(System.out::println)</code>
<code>.boxed()</code>	Wandelt primitive Werte in Wrapper-Objekte um	<code>IntStream</code>	<code>Stream<Integer></code>	<code>IntStream.of(1,2).boxed() → [1,2]</code>

Terminal Operations

Operation	Beschreibung	Eingabe	Ausgabe	Beispiel
<code>.toList()</code>	Gibt alle Elemente als unveränderbare Liste zurück	<code>Stream<T></code>	<code>List<T></code> (unmodifiable)	<code>Stream.of(1,2).toList() → [1,2]</code>
<code>.collect()</code>	Wandelt Stream in beliebige Sammlung um	<code>Stream<T></code>	<code>Collection<T></code> (z. B. Set, List, Map)	<code>stream.collect(Collectors.toSet())</code>
<code>.reduce()</code>	Kombiniert alle Werte zu einem Ergebnis	<code>Stream<Integer></code>	<code>Integer</code>	<code>Stream.of(1,2,3).reduce(0, Integer::sum) → 6</code>
<code>.forEach()</code>	Führt Aktion für jedes Element aus (kein Rückgabewert)	<code>Stream<T></code>	<code>void</code>	<code>.forEach(System.out::println)</code>
<code>.count()</code>	Zählt Elemente im Stream	<code>Stream<T></code>	<code>long</code>	<code>Stream.of("A", "B").count() → 2</code>
<code>.anyMatch()</code>	Prüft, ob mind. ein Element Bedingung erfüllt	<code>Stream<T></code>	<code>boolean</code>	<code>.anyMatch(x -> x>5) → true</code>
<code>.allMatch()</code>	Prüft, ob alle Bedingung erfüllen	<code>Stream<T></code>	<code>boolean</code>	<code>.allMatch(x -> x>0) → true</code>
<code>.noneMatch()</code>	Prüft, ob kein Element Bedingung erfüllt	<code>Stream<T></code>	<code>boolean</code>	<code>.noneMatch(x -> x<0) → true</code>
<code>.findFirst()</code>	Gibt erstes Element (optional) zurück	<code>Stream<T></code>	<code>Optional<T></code>	<code>.findFirst() → Optional[1]</code>
<code>.min() / .max()</code>	Liefert kleinstes/größtes Element	<code>Stream<T></code>	<code>Optional<T></code>	<code>.min(Comparator.naturalOrder())</code>

Primitive Streams und Optional

Typ / Methode	Beschreibung	Eingabe	Ausgabe	Beispiel
<code>IntStream,</code> <code>LongStream,</code> <code>DoubleStream</code>	Spezielle Streams für primitive Werte	primitive Werte (int, long, double)	Stream entsprechender Typen	<code>IntStream.range(1, 4) → 1, 2, 3</code>
<code>.sum()</code>	Addiert alle Werte	<code>IntStream</code>	<code>int</code>	<code>IntStream.of(1, 2, 3).sum() → 6</code>
<code>.average()</code>	Berechnet Durchschnitt	<code>IntStream</code>	<code>OptionalDouble</code>	<code>IntStream.of(2, 4, 6).average() → OptionalDouble[4.0]</code>
<code>.boxed()</code>	Verpackt primitive Werte als Objekte	<code>IntStream</code>	<code>Stream<Integer></code>	<code>IntStream.of(1, 2, 3).boxed() → [1, 2, 3]</code>
<code>Optional<T></code>	Verpackt evtl. leere Werte	evtl. null	<code>Optional<T></code>	<code>Optional.ofNullable(value)</code>
<code>.orElse()</code>	Gibt Wert oder Default-Wert	<code>Optional<T></code>	<code>T</code>	<code>optional.orElse("default")</code>
<code>.orElseThrow()</code>	Gibt Wert oder Exception	<code>Optional<T></code>	<code>T</code> oder Exception	<code>optional.orElseThrow()</code>